

A COMPARISON OF AUDIO SIGNAL PREPROCESSING METHODS FOR DEEP NEURAL NETWORKS ON MUSIC TAGGING

Keunwoo Choi, György Fazekas, Mark Sandler*

Queen Mary University of London, London, UK
Centre for Digital Music, EECS
E1 4FZ, London, UK
keunwoo.choi@qmul.ac.uk

Kyunghyun Cho†

New York University, NY, USA
Center for Data Science
715 Broadway, New York, NY, USA
kyunghyun.cho@nyu.edu

ABSTRACT

In this paper, we empirically investigate the effect of audio preprocessing on music tagging with deep neural networks. We perform comprehensive experiments involving audio preprocessing using different time-frequency representations, logarithmic magnitude compression, frequency weighting, and scaling. We show that many commonly used input preprocessing techniques are redundant except magnitude compression.

1. INTRODUCTION

Many music information retrieval researches using deep learning usually focus on optimising the hyperparameters which specify the network structure. Meanwhile, the audio preprocessing stage is often heuristically decided and not subject to optimisation.

Although neural networks are known to be universal function approximators [1], training efficiency and performance may vary significantly with different training methods as well as generic techniques including preprocessing the input data [2]. In other words, a neural network can *represent* any function but it does not mean it can effectively *learn* any function. Therefore, both empirical decisions and domain knowledge are crucial since choosing between various preprocessing methods can be seen as a non-differentiable choice function, therefore it cannot be optimised using gradient-based learning methods.

For example, mel-spectrograms have been preferred over short-time Fourier transform in many tasks [3] because it was considered to have enough information despite of its smaller size, i.e., efficient yet effective. When a time-frequency representation $\mathbf{X}$ is given, one of the most common preprocessing

approaches is to apply logarithmic compression, i.e., $\log(\mathbf{X} + \alpha)$ where α can be arbitrary constants such as very small number (e.g. 10^{-7}) or 1. However, the performances of these methods are usually not strictly compared.

In this paper, we focus on audio preprocessing strategies for deep convolutional neural networks for music tagging. By providing empirical results with various preprocessing strategies, we aim to demystify the effect of audio preprocessing methods on the performances. This will help researchers design deep learning systems for music research.

2. EXPERIMENTS AND DISCUSSIONS

A representative network structure needs to be defined to compare the effects of audio preprocessing. A ConvNet (convolutional neural networks) with 2D kernels and 2D convolution axes was chosen. This showed a good performance with efficient training in a prior benchmark [4], where the model we selected was denoted *k2c2*, indicating 2D kernels and convolution axes. As illustrated in Figure 1, homogeneous 2D (3×3) convolutional kernels are used in every convolutional layer. The input has a single channel, 96 mel bins, and 1,360 temporal frames, denoted as (1, 96, 1360). The figures in Table 1 denote the number of channels (N), kernel height and kernel width for convolutional layers and subsampling height, subsampling width for max-pooling layers. Here, the height and width corresponds to the frequency- and time-axes respectively. Exponential linear unit (ELU) is used as an activation function in all convolutional layers [5].

For the training of music tagger, we used the Million Song Dataset (MSD) [6] with preview audio clips. The training data are 30-second stereo mp3 files with a sampling rate of 22,050Hz and 64 kbps constant bit-rate encoding. For efficient training in our experiments, we downmix and downsample the signals to 12 kHz after decoding and trim the audio duration to 29-second to ensure equal-sized input signals. The short-time Fourier transform and melspectrogram are computed using a hop size of 256 samples (21.3 ms) with a 512-point discrete Fourier transform aggregated to yield 96 mel bins per frame. The preprocessing is performed using *Librosa*

*FAST IMPACT EPSRC Grant EP/L019981/1 and the European Commission H2020 research and innovation grant AudioCommons (688382). Mark Sandler acknowledges the support of the Royal Society as a recipient of a Wolfson Research Merit Award.

†Kyunghyun Cho thanks the support by Facebook, Google (Google Faculty Award 2016) and NVidia (GPU Center of Excellence 2015-2016)

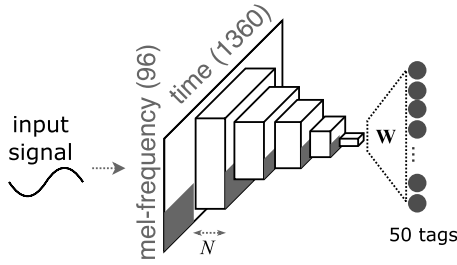


Fig. 1: Network structure of the 5-layer ConvNet. N refers to the number of feature maps (which is set to 32 for all layers in this paper) while $\mathbf{W}$ refers to the weights matrix of the fully-connected output layer.)

Table 1: Details of the ConvNet architecture shown in Figure 1. 2-dimensional convolutional layer is specified by (channel, (kernel lengths in frequency, time)). Pooling layer is specified by (pooling length in frequency and time)

<i>input (1, 96, 1360)</i>	
Conv2d and Max-Pooling	(32, (3, 3)) and (2, 4)
Conv2d and Max-Pooling	(32, (3, 3)) and (4, 4)
Conv2d and Max-Pooling	(32, (3, 3)) and (4, 5)
Conv2d and Max-Pooling	(32, (3, 3)) and (2, 4)
Conv2d and Max-Pooling	(32, (3, 3)) and (4, 4)
Fully-connected layer	(50)
<i>output (50)</i>	

[7] and Kapre [8]. Total 224,242 tracks are used and split into train/validation/test sets, 201,672/12,633/28,537 tracks respectively.¹ During training, the binary cross-entropy function is used as a loss function. For the acceleration of stochastic gradient descent, we use adaptive optimisation based on Adam [9]. The experiment is implemented in Python with *Keras* [10] and *Theano* [11] as deep learning frameworks.

In the all experiments, area under curve - of receiver operating characteristic (AUC) is used as a metric. Although it can be lower than 0.5 in theory, AUC practically ranges in [0.5, 1.0] as random and perfect predictions show 0.5 and 1.0 of AUC respectively. The reported AUC scores are all measured on the test set.

2.1. Variance by different initialisation

In deep learning, using K -fold cross-validation is not a standard practice for two reasons. First, with large enough data and a good split of train, validation and test sets, the model can be trained with small variance. Second, the cost of hyperparameter search is very high and it makes repeating experiments too expensive in practice. For these reasons, we do not cross-validate the ConvNet in this study. Instead, we present

¹The network implementation and split setting are online:

https://github.com/keunwoochoi/transfer_learning_music and
https://github.com/keunwoochoi/MSD_split_for_tagging

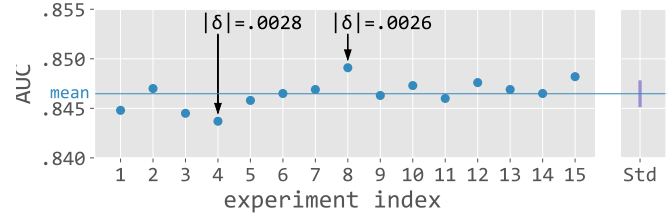


Fig. 2: AUC scores (left), their mean, and the standard deviation of the AUC scores (right). The two deltas on the plot indicate the difference between the average AUC and the scores of experiments 4 and 8.

the results of repeated experiments with fixed network and training hyperparameters, such as training example sequences and batch size. This experiment therefore measures the variance of the model introduced by different weight initialisations of the convolutional layers. For this, a normal distribution is used following He et al. [12], which has been shown to yield a stable training procedure.

The results are summarised in Figure 2. This shows the AUC scores of 15 repeated experiments on the left as well as their standard deviation on the right. Small standard deviation indicates that we can obtain a reliable, precise score by repeating the same experiments for a sufficient number of times. The two largest differences observed between the average AUC score and that of experiment 4 and 8 (AUC differences of 0.0028 and 0.0026 respectively) indicate that we may obtain up to ~ 0.005 AUC difference among experiment instances. Based on this, we can assume that an AUC difference of < 0.005 is non-significant in this paper.

2.2. Time-frequency representations

STFT and melspectrogram have been the most popular input representations for music classification [3]. Although sample-based deep learning methods have been introduced, 2-dimensional representations would be still useful in the near future for efficient training. Melspectrograms provide an efficient and perceptually relevant representation compared to STFT [13] and have been shown to perform well in various tasks [14–18]. However, an STFT is closer to the original signal and neural networks may be able to learn a representation that is more optimal to the given task than mel-spectrograms. This requires large amounts of training data however, as reported in [18] where using melspectrograms outperformed STFT with a smaller dataset.

Figure 3 shows the AUC scores obtained using log(STFT) vs. log(melspectrogram) while varying the size of the utilised training data. Although there are small differences on AUC scores up to 0.007, neither of them outperforms the other, especially when enough data is provided. This rebuts a previous result in [18] because melspectrograms did not have a clear advantage here even with a small training data size. This may be due to the difference in frequency resolution of the representations used and summarised as follows.

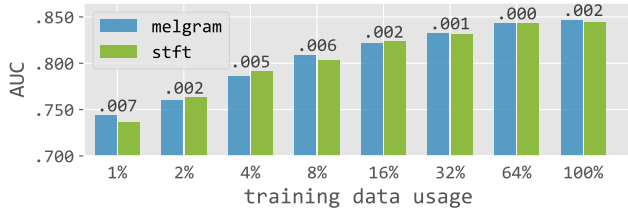


Fig. 3: Performances of predictions with melspectrogram and STFT with varying training data sizes. The numbers above bars indicate the absolute performance differences between melspectrograms and STFTs.

- STFT in [18]: $6000/129=46.5$ Hz (256-point FFT with 12 kHz sampling rate)
- STFT in our work: $6000/257=23.3$ Hz (512-point FFT with 12 kHz sampling rate)
- Melspectrogram in [18] and our work: 35.9 Hz for frequency < 1 kHz (96 mel-bins and by [19] and [7])

In [18], the frequency resolution of the STFT was lower than that of the melspectrogram to enable comparing them with similar number of frequency bins. On the contrary, STFT of higher frequency resolution is used in our experiment. Nevertheless, it is found to be only as good as melspectrogram in terms of performance, i.e., the network does not take advantage of finer input. This means overall that, in practice, melspectrogram may be preferred since its smaller size leads to reduced computation in training and prediction. The figure also illustrates how much the data size affects the performance. Exponentially increasing data size merely results in a linear AUC improvement. AUC starts to converge at 64% and 100%.

2.3. Analysis of scaling effects and frequency-axis weights

In this section, we discuss the effects of magnitude manipulation. Preliminary experiments suggested that there might be two independent aspects to investigate; *i*) frequency-axis weights and *ii*) magnitude scaling of each item in the training set. Examples of frequency-axis weights are illustrated in Figure 4, where different weighting schemes are plotted. Our experiment is designed to isolate these two effects. We tested two input representations *log-melspectrogram* vs. *mel-spectrogram*, with three frequency weighting schemes *per-frequency*, *A-weighting* and *bypass*, as well as two scaling methods $\times 10$ (on) and $\times 1$ (off), yielding $2 \times 3 \times 2 = 12$ configurations in total. We summarise the mechanism of each block as follows. First, there are three frequency weights schemes.

- *per-frequency stdd*: Often called spectral whitening. Compute means and standard deviations across time, i.e., per-frequency, and standardise each frequency band using these values. The average frequency response becomes flat (equalised). This method has been used in the

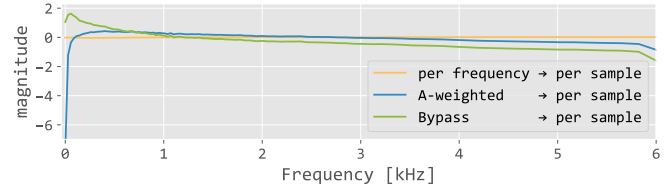


Fig. 4: Average frequency magnitude of randomly selected 200 excerpts with three frequency-axis normalisation. A per-sample (excerpt) standardisation follows to remove the effect of different overall average magnitude.

literature for tagging [18], singing voice detection [20] and transcription [21].

- *A-weighted*: Apply the international standard IEC 61672:2003 A-weighting curve, which approximates human perception of loudness as a function of frequency.
- *Bypass*: Do not apply any processing, i.e., $f : \mathbf{X} \rightarrow \mathbf{X}$

There are two other blocks which are mutually independent and also independent to frequency weights schemes.

- *per-sample stdd*: Excerpt-wise normalisation with its overall mean and standard deviation, i.e., using statistics across time and frequency of each spectrogram.
- $\times 10$ *scaler*: Multiply the input spectrogram by 10, i.e., $f : \mathbf{X} \rightarrow 10\mathbf{X}$.

2.3.1. Frequency weighting

This process is related to the loudness, i.e., human perception of sound energy [13], which is a function of frequency. The sensitivity of the human auditory system drops substantially below a few hundred Hz², hence music signals typically exhibit higher energy in the lower range to compensate for the attenuation. This is illustrated in Figure 4, where uncompensated average energy measurements corresponding to the *Bypass* curve (shown in green) yield a peak at low frequencies. This imbalance affects neural network activations in the first layer which may influence performance. To assess this effect, we tested three frequency weighting approaches. Their typical profiles are shown in Figure 4. In all three strategies, excerpt-wise standardisation is used to alleviate scaling effects (see Section 2.3.2).

Our test results show that networks using the three strategies all achieve similar AUC scores. The performance differences within four groups, $\{1, 1s, 2, 2s\}$ in Figure 5 are small and none of them are governing the others. The curves in Figure 4 show the average input magnitudes over frequency. These offsets change along frequency, but the change does not seem large enough to corrupt the local patterns due to the locality of ConvNets, and therefore the network is learning useful representations without significant performance differences within each group.

²See equal-loudness contours e.g. in ISO 226:2003.

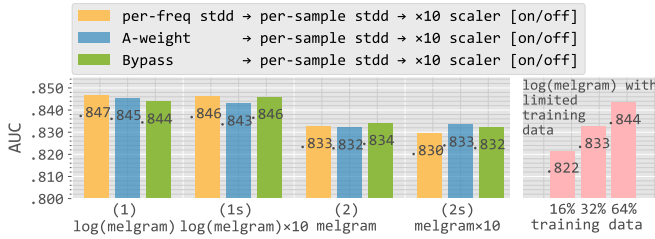


Fig. 5: Performance comparisons of different preprocessing procedures. From left to right, four groups of scores are from different magnitude processing (melspectrogram in decibel scale and linear scale), with additional $\times 10$ scaler turned on/off. In each group, yellow/blue/green bars indicates different frequency-axis processing as annotated in the legend. Logarithmic compression is done before other preprocessings are applied.

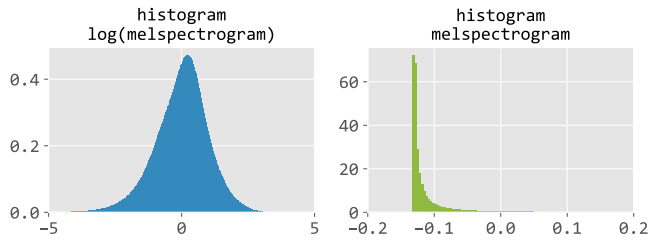


Fig. 6: histograms of the magnitude of melspectrogram time-frequency bins with (left) and without (right) logarithmic compression. The number of bins are 100 and both are normalised, i.e., $\sum_{i=1}^{100} 0.01 \times y_i = 1$. Log compression significantly affects the histogram, making the distribution Gaussian (left), otherwise extremely skewed (right). This is after standardisation and based on randomly selected 100 tracks from the training set.

2.3.2. Analysis of scaling effects

We may assume a performance increase if we scale the overall magnitudes for a number of reasons. During training using gradient descent, the gradient of error with respect to weights $\frac{\partial E}{\partial W}$ is proportional to $\frac{\partial}{\partial W} f(W^T X)$ where f is the activation function. This means that the learning rate of a layer is proportional to the magnitude of input X . In particular, the first layer usually receives the weakest error backpropagation, hence scaling of the input may affect the overall performance.

We tested the effect of this with the results shown in Fig. 5. To this end, consider comparing the same-coloured bars of $\{1 \text{ vs. } 1s\}$ and $\{2 \text{ vs. } 2s\}$. Here, the scaling factor is set to 10 for illustration, however many possible values < 100 were tested and showed similar results. In summary, this hypothesis is rebutted – scaling did not affect the performance. The analysis of trained weights revealed that different magnitudes of the input only affects the bias of the first convolutional layer. Training with scaling set to $\times 10$ results in 3.4 times larger mean absolute value of the biases in the first layer. This is due to batch normalization [22] which compensates for the different magnitudes by normalizing the activations of convolutional layers.

2.4. Log-compression of magnitudes

Lastly, we discuss how logarithmic compression of magnitudes, i.e. decibel scaling, affects performance. This is considered standard preprocessing in music information retrieval. The procedure is motivated by the human perception of loudness [13] which has logarithmic relationship with the physical energy of sound. Although learning a logarithmic function is a trivial task for neural networks, it can be difficult to implicitly learn an optimal nonlinear compression when it is embedded in a complicated task. A nonlinear compression was also shown to affect the performance in visual image recognition using neural networks [23]. Figure 6 compares the histograms of the magnitudes of time-frequency bins after zero-mean unit-variance standardisation. On the left, a logarithmically compressed melspectrogram shows an approximately Gaussian distribution without any extreme values. Meanwhile, the bins of linear melspectrogram on the right is extremely condensed in a very small range while they range in wider region overall. This means the network should be trained with higher numerical precision to the input, hence more vulnerable to noise.

As a result, decibel-scaled melspectrograms always outperform the linear versions as shown in Fig 5, where the same-coloured bars should be compared across within $\{1 \text{ vs. } 2\}$ and $\{1s \text{ vs. } 2s\}$. Colours indicate normalization schemes while $\{1 \text{ vs. } 1s\}$ and $\{2 \text{ vs. } 2s\}$ compare effect, both of which are explained in Section 2.3³. Compared to the performance differences while controlling the training set size (the pink bar charts on the right of Figure 5) the additional work introduced by not using decibel scaling can be roughly estimated by comparing these scores to those networks when the training data size is limited (in pink). While this also depends on other configurations of the task, seemingly twice the data is required to compensate for the disadvantage of not using a decibel scaled representation.

3. CONCLUSION

In this paper, we have shown that some of the input preprocessing methods can affect the performance. We quantify this in terms of the size of the training data required to achieve similar performances. Among several preprocessing techniques tested in this study, only logarithmic scaling of the magnitude resulted in significant improvement. In other words, the network was resilient to most modifications of the input data except logarithmic compression of magnitudes in various time-frequency representations. Although we focused on the music tagging task, our results provide general knowledge applicable in many similar machine-listening problems, e.g., music genre classification or the prediction of environmental sound descriptors.

³Decibel-scaled STFT also outperformed linear STFT in our unreported experiments.

4. REFERENCES

- [1] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural networks*, vol. 4, no. 2, pp. 251–257, 1991.
- [2] Y. A. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Neural networks: Tricks of the trade*. Springer, 2012, pp. 9–48.
- [3] K. Choi, G. Fazekas, K. Cho, and M. Sandler, "A tutorial on deep learning for music information retrieval," *arXiv preprint arXiv:1709.04396*, 2017.
- [4] K. Choi, G. Fazekas, M. Sandler, and K. Cho, "Convolutional recurrent neural networks for music classification," in *2017 IEEE International Conference on Acoustics, Speech, and Signal Processing*, 2017.
- [5] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units," *arXiv:1511.07289*, 2015.
- [6] T. Bertin-Mahieux, D. P. Ellis, B. Whitman, and P. Lamere, "The million song dataset," in *ISMIR 2011: Proceedings of the 12th International Society for Music Information Retrieval Conference, October 24-28, 2011, Miami, Florida*. University of Miami, 2011, pp. 591–596.
- [7] B. McFee, M. McVicar, O. Nieto, S. Balke, C. Thome, D. Liang, E. Battenberg, J. Moore, R. Bittner, R. Yamamoto, D. Ellis, F.-R. Stoter, D. Repetto, S. Waloschek, C. Carr, S. Kranzler, K. Choi, P. Viktorin, J. F. Santos, A. Holovaty, W. Pimenta, and H. Lee, "librosa 0.5.0," Feb. 2017. [Online]. Available: <https://doi.org/10.5281/zenodo.293021>
- [8] K. Choi, D. Joo, and J. Kim, "Kapre: On-gpu audio preprocessing layers for a quick implementation of deep neural network models with keras," in *Machine Learning for Music Discovery Workshop at 34th International Conference on Machine Learning*, 2017.
- [9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization." 3rd International Conference for Learning Representations, San Diego, 2015, 2014. [Online]. Available: <http://arxiv.org/abs/1412.6980>
- [10] F. Chollet, "Keras: Deep learning library for theano and tensorflow," <https://github.com/fchollet/keras>, 2015.
- [11] F. Bastien, P. Lamblin, R. Pascanu, J. Bergstra, I. Goodfellow, A. Bergeron, N. Bouchard, D. Warde-Farley, and Y. Bengio, "Theano: new features and speed improvements," *arXiv preprint arXiv:1211.5590*, 2012.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the IEEE international conference on computer vision*, 2015.
- [13] B. C. Moore, *An introduction to the psychology of hearing*. Brill, 2012.
- [14] A. Van den Oord, S. Dieleman, and B. Schrauwen, "Deep content-based music recommendation," in *Advances in Neural Information Processing Systems*, 2013, pp. 2643–2651.
- [15] S. Dieleman and B. Schrauwen, "End-to-end learning for music audio," in *Acoustics, Speech and Signal Processing (ICASSP), 2014 IEEE International Conference on*. IEEE, 2014, pp. 6964–6968.
- [16] J. Schluter and S. Bock, "Improved musical onset detection with convolutional neural networks," in *Acoustics, Speech and Signal Processing, IEEE International Conference on*. IEEE, 2014.
- [17] K. Ullrich, J. Schlüter, and T. Grill, "Boundary detection in music structure analysis using convolutional neural networks," in *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR 2014), Taipei, Taiwan*, 2014.
- [18] K. Choi, G. Fazekas, and M. Sandler, "Automatic tagging using deep convolutional neural networks," in *The 17th International Society of Music Information Retrieval Conference, New York, USA*. ISMIR, 2016.
- [19] M. Slaney, "Auditory toolbox," *Interval Research Corporation, Tech. Rep*, vol. 10, p. 1998, 1998.
- [20] J. Schlüter, "Learning to pinpoint singing voice from weakly labeled examples," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2016, pp. 44–50.
- [21] S. Sigtia, E. Benetos, and S. Dixon, "An end-to-end neural network for polyphonic music transcription," *arXiv preprint arXiv:1508.01774*, 2015.
- [22] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *arXiv preprint arXiv:1502.03167*, 2015.
- [23] S. F. Dodge and L. J. Karam, "Understanding how image quality affects deep neural networks," *CoRR*, vol. abs/1604.04004, 2016. [Online]. Available: <http://arxiv.org/abs/1604.04004>